

SECOND EDITION

Deep Learning with PyTorch

Training and applying deep learning
and generative AI models

Howard Huang
Eli Stevens
Luca Antiga
Thomas Viehmann

 MANNING



Praise for the first edition

With this publication, we finally have a definitive treatise on PyTorch. It covers the basics and abstractions in great detail.

—From the Foreword by Soumith Chintala, Cocreator of PyTorch

Deep learning divided into digestible chunks with code samples that build up logically.

—Mathieu Zhang, NVIDIA

Timely, practical, and thorough. Don't put it on your bookshelf but next to your laptop.

—Philippe Van Bergen, P2 Consulting

Deep Learning with PyTorch offers a very pragmatic overview of deep learning. It is a didactical resource.

—Orlando Alejo Mendez Morales, Experian



Deep Learning with PyTorch, Second Edition

Luca Antiga, Eli Stevens, Howard Huang, Thomas Viehmann

To comment go to [livebook](#).



Manning
Shelter Island

For more information on this and other Manning titles go to [manning.com](#).

copyright

For online information and ordering of this and other Manning books, please visit www.manning.com. The publisher offers discounts on this book when ordered in quantity. For more information, please contact

Special Sales Department

Manning Publications Co.

20 Baldwin Road

PO Box 761

Shelter Island, NY 11964

Email: orders@manning.com

©2026 by Manning Publications Co. All rights reserved.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by means electronic, mechanical, photocopying, or otherwise, without prior written permission of the publisher.

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in the book, and Manning Publications was aware of a trademark claim, the designations have been printed in initial caps or all caps.

 Recognizing the importance of preserving what has been written, it is Manning's policy to have the books we publish printed on acid-free paper, and we exert our best efforts to that end. Recognizing also our responsibility to conserve the resources of our planet, Manning books are printed on paper that is at least 15 percent recycled and processed without the use of elemental chlorine.

The authors and publisher have made every effort to ensure that the information in this book was correct at press time. The authors and publisher do not assume and hereby disclaim any liability to any party for any loss, damage, or disruption caused by errors or omissions, whether such errors or omissions result from negligence, accident, or any other cause, or from any usage of the information herein.

Manning Publications Co.

20 Baldwin Road

PO Box 761

Shelter Island, NY 11964

Development editor: Elesha Hyde
Technical editor: Fábio Vinicius Moreira Perez
Review editor: Angelina Lazukić
Production editor: Aleksandar Dragosavljević
Copy editor: Alisa Larson
Proofreader: Mike Beady
Typesetter: Dennis Dalinnik
Cover designer: Marija Tudor

ISBN: 9781633438859

Printed in the United States of America

dedication

To Fred, Mary, Ward, and Edward, the best unpaid editors, critics, and supporters I could ask for

—Howard Huang

Same :-) But, really, this is for you, Alice and Luigi

—Luca Antiga

To my wife (this book would not have happened without her invaluable support and partnership), my parents (I would not have happened without them), and my children (this book would have happened a lot sooner but for them), thank you for being my home, my foundation, and my joy

—Eli Stevens

To Eva, Rebekka, Jonathan, and David

—Thomas Viehmann

contents

[preface](#)

[acknowledgments](#)

[about this book](#)

[about the authors](#)

[about the cover illustration](#)

[Part 1 Core PyTorch](#)

Evaluation Only. Created with Aspose.Words. Copyright 2003-2026 Aspose Pty Ltd.

1 Introducing deep learning and the PyTorch library

[1.1 What is deep learning?](#)

[1.2 The shift from machine learning to deep learning](#)

[1.3 What to expect](#)

[1.4 Why PyTorch?](#)

[1.4.1 The deep learning competitive landscape](#)

[1.5 How PyTorch supports deep learning projects](#)

[1.6 Hardware and software requirements](#)

[1.6.1 Using Jupyter Notebooks](#)

[1.7 Exercises](#)

2 Pretrained networks

[2.1 A pretrained network that recognizes the subject of an image](#)

[2.1.1 Obtaining a pretrained network for image recognition](#)

[2.1.2 AlexNet](#)

[2.1.3 The Vision Transformer](#)

[2.1.4 Ready, set, almost run](#)

[2.1.5 Run!](#)

[2.2 Generating and editing images](#)

[2.2.1 The inpainting process](#)

[2.2.2 A network that turns horses into zebras](#)

[2.3 Model Zoo: Hugging Face](#)

[2.4 A pretrained network that describes scenes](#)

[2.4.1 BLIP in action](#)

[2.5 Conclusion](#)

[2.6 Exercises](#)

3 It starts with a tensor

[3.1 The world as floating-point numbers](#)

[3.2 Tensors: Multidimensional arrays](#)

[3.2.1 From Python lists to PyTorch tensors](#)

[3.2.2 Constructing our first tensors](#)

[3.2.3 The essence of tensors](#)

[3.3 Indexing tensors](#)

[3.4 Broadcasting](#)

[3.5 Named tensors](#)

[3.6 Tensor element types](#)

[3.6.1 Specifying the numeric type with dtype](#)

[3.6.2 A dtype for every occasion](#)

[3.6.3 Managing a tensor's dtype attribute](#)

[3.7 The tensor API](#)

[3.8 Tensors: Scenic views of storage](#)

[3.8.1 Indexing into storage](#)

[3.8.2 Modifying stored values: In-place operations](#)

[3.9 Tensor metadata: Size, offset, and stride](#)

[3.9.1 Views of another tensor's storage](#)

[3.9.2 Transposing without copying](#)

[3.9.3 Transposing in higher dimensions](#)

[3.9.4 Contiguous tensors](#)

[3.10 Moving tensors to the GPU](#)

[3.10.1 Managing a tensor's device attribute](#)

[3.11 NumPy interoperability](#)

[3.12 Generalized tensors are tensors, too](#)

[3.13 Serializing tensors](#)

[3.13.1 Serializing to HDF5 with h5py](#)

[3.14 Conclusion](#)

[3.15 Exercises](#)

4 Real-world data representation using tensors

[4.1 Working with images](#)

[4.1.1 Adding color channels](#)

[4.1.2 Loading an image file](#)

[4.1.3 Changing the layout](#)

[4.1.4 Normalizing the data](#)

[4.2 3D images: Volumetric data](#)

[4.2.1 Loading a specialized format](#)

[4.3 Representing tabular data](#)

[4.3.1 Using a real-world dataset](#)

[4.3.2 Loading a wine data tensor](#)

[4.3.3 Representing scores](#)

[4.3.4 One-hot encoding](#)

[4.3.5 When to categorize](#)

[4.3.6 Finding thresholds](#)

[4.4 Working with time series](#)

[4.4.1 Adding a time dimension](#)

[4.4.2 Shaping the data by time period](#)

[4.4.3 Ready for training](#)

[4.5 Representing text](#)

[4.5.1 Converting text to numbers](#)

[4.5.2 One-hot-encoding characters](#)

[4.5.3 One-hot encoding whole words](#)

[4.5.4 Text embeddings](#)

[4.5.5 Text embeddings as a blueprint](#)

[4.6 Conclusion](#)

[4.7 Exercises](#)

[5 The mechanics of learning](#)

[5.1 A timeless lesson in modeling](#)

[5.2 Learning is just parameter estimation](#)

[5.2.1 A hot problem](#)

[5.2.2 Gathering some data](#)

[5.2.3 Visualizing the data](#)

[5.2.4 Choosing a linear model as a first try](#)

[5.3 Less loss is what we want](#)

[5.3.1 From problem back to PyTorch](#)

[5.4 Down along the gradient](#)

[5.4.1 Decreasing loss](#)

[5.4.2 Getting analytical](#)

[5.4.3 Iterating to fit the model](#)

[5.4.4 Normalizing inputs](#)

[5.4.5 Visualizing \(again\)](#)

[5.5 PyTorch's autograd: Backpropagating all things](#)

[5.5.1 Computing the gradient automatically](#)

[5.5.2 Optimizers à la carte](#)

[5.5.3 Training, validation, and overfitting](#)

[5.5.4 Training set](#)

[5.5.5 Autograd nits and switching it off](#)

[5.6 Conclusion](#)

[5.7 Exercises](#)

[6 Using a neural network to fit the data](#)

[6.1 Artificial neurons](#)

[6.1.1 Composing a multilayer network](#)

[6.1.2 Understanding the error function](#)

[6.1.3 Adding nonlinearity with activation functions](#)

[6.1.4 More activation functions](#)

[6.1.5 Choosing the best activation function](#)

[6.1.6 What learning means for a neural network](#)

[6.2 The PyTorch nn module](#)

[6.2.1 Using nn.Module as a callable](#)

[6.2.2 Returning to the linear model](#)

[6.3 Finally, a neural network](#)

[6.3.1 Replacing the linear model](#)

[6.3.2 Inspecting the parameters](#)

[6.3.3 Comparing to the linear model](#)

[6.4 Conclusion](#)

[6.5 Exercises](#)

[7 Telling birds from airplanes: Learning from images](#)

[7.1 A dataset of tiny images](#)

[7.1.1 Downloading CIFAR-10](#)

[7.1.2 The Dataset class](#)

[7.1.3 Dataset transforms](#)

[7.1.4 Normalizing data](#)

[7.2 Distinguishing birds from airplanes](#)

[7.2.1 Building the dataset](#)

[7.2.2 A fully connected model](#)

[7.2.3 Output of a classifier](#)

[7.2.4 Representing the output as probabilities](#)

[7.2.5 Training the classifier](#)

[7.2.6 The limits of going fully connected](#)

[7.3 Conclusion](#)

[7.4 Exercises](#)

[8 Using convolutions to generalize](#)

[8.1 The case for convolutions](#)

[8.1.1 What convolutions do](#)

[8.2 Convolutions in action](#)

[8.2.1 Padding the boundary](#)

[8.2.2 Detecting features with convolutions](#)

[8.2.3 Looking further with depth and pooling](#)

[8.2.4 Putting it all together for our network](#)

[8.3 Subclassing nn.Module](#)

[8.3.1 Our network as an nn.Module](#)

[8.3.2 How PyTorch keeps track of parameters and submodules](#)

This document was truncated here because it was created in the Evaluation Mode.